

Online Game level Editor Maze Graph Using CRUD Operations

M.ShivaNandheswarareddy
CDS/2025/1801
School of Computing and Data Science
Sai University
J.Kalyan Ram
CDS/2025/1800
School of Computing and Data Science
Sai University

1. Abstract

This project presents the development of a Maze Level Editor using Graph Data Structure implemented in the C programming language. The system models a maze as a graph where rooms are represented as nodes and paths between them as edges. The project supports dynamic manipulation of the maze using CRUD (Create, Read, Update, Delete) operations through a menu-driven interface.

The adjacency list representation is used for efficient storage and traversal. Users can create rooms, connect paths, update connections, delete elements, and search within the maze. The system demonstrates practical application of data structures such as graphs and linked lists along with dynamic memory allocation.

2. Introduction

Maze-based games require flexible level design systems to create and manage paths and rooms efficiently. Traditional static designs lack adaptability and scalability.

This project introduces a graph-based maze editor where:

- Each room is a node
- Each path is an edge

Using a graph allows dynamic changes and efficient representation of complex maze structures. The project is implemented in C to strengthen understanding of low-level memory management and data structures.

3. Objectives

The main objectives of this project are:

- To implement a graph using adjacency list representation
 - To design a maze level editor using graph concepts
 - To perform CRUD operations on graph elements
 - To enable dynamic memory handling using `malloc()` and `free()`
 - To provide a menu-driven interface for user interaction
 - To implement search functionality within the maze
-

4. Scope of the Project

This project is limited to:

- Text-based maze editing using terminal interface
- Representation of maze using graph only (no GUI)
- Basic operations like adding, deleting, updating nodes and edges

Limitations (don't ignore this — examiners care):

- No graphical visualization of the maze
- No pathfinding algorithms like BFS/DFS implemented fully
- No file storage (data is lost after execution)

Where it *could* be used:

- Educational tools for learning graph structures
 - Basic game logic prototypes
-

5. Literature Review

Graphs are widely used in computer science to model relationships and connections. Maze problems are classic applications of graph theory.

Existing systems:

- Maze generators use DFS or Prim's Algorithm
- Game engines use advanced graph structures for navigation

This project differs because:

- It focuses on manual level editing
 - It emphasizes CRUD operations on graphs
 - It is implemented in pure C, unlike modern tools using high-level languages
-

6. Methodology

The project follows a structured procedural approach:

Step 1: Graph Initialization

Create an empty graph using adjacency list

Step 2: User Interaction

Menu-driven system allows user to select operations

Step 3: CRUD Operations

- Add rooms (nodes)
- Add paths (edges)
- Update connections
- Delete rooms/paths

Step 4: Traversal & Search

Traverse adjacency list to display or locate nodes

Step 5: Memory Management

Allocate and deallocate memory dynamically

7. Technology Used

- Programming Language: C
 - Compiler: GCC
 - Platform: Command Line / Terminal
 - Concepts Used:
 - Graph Data Structure
 - Linked List
 - Dynamic Memory Allocation
 - Pointers
-

8. Architecture/System Design

Core Components:

1. Graph Structure
 - Array of linked lists
 - Each index represents a room
2. Node Structure
 - Stores room ID
 - Pointer to next connected room
3. Adjacency List
 - Stores connections for each node

Flow:

User Input → Menu → Operation Selection
→ Graph Function → Output Display

Internal Working:

- Nodes are stored dynamically
 - Edges are added bidirectionally (if undirected)
 - Linked lists maintain connections
-

9. Implementation Details

Key Functionalities:

1. Add Room (Node)
 - Allocate memory
 - Insert into graph
2. Add Path (Edge)
 - Connect two nodes
 - Update adjacency list
3. Display Maze
 - Traverse all adjacency lists
 - Print connections
4. Update
 - Modify node or connection
5. Delete
 - Remove node or edge
 - Free memory
6. Search
 - Traverse adjacency list to find node

Compilation:

```
gcc maze.c -o maze
./maze
```

10. Results / Output

The program successfully performs:

- Creation of maze structure
- Dynamic addition and deletion of rooms and paths
- Display of adjacency list representation
- Search functionality

Example Output:

- Rooms and their connections displayed clearly
- Menu-driven operations executed correctly

👉 Reality check:

Your “results” are basic. There’s no performance metrics, no test cases, no complexity analysis — that’s a weak spot.

11. Conclusion

The Maze Level Editor demonstrates the practical use of graph data structures in designing dynamic systems. The implementation of CRUD operations allows flexible manipulation of maze elements.

The project strengthens understanding of:

- Graph representation
- Linked list usage
- Memory management in C

However, the system remains a basic prototype and lacks advanced features required for real-world applications.

12. Future Work

Here’s where you should’ve pushed harder:

- Add GUI for visual maze editing
- Implement BFS/DFS for pathfinding
- Add file storage (save/load maze)
- Introduce weighted edges (for difficulty levels)

- Convert to real-time game engine integration
-

13. Footnotes / References

1. Data Structures using C – Reema Thareja
2. Introduction to Algorithms – Thomas H. Cormen
3. Graph Theory Concepts – NPTEL Lectures
4. GCC Compiler Documentation
5. Standard C Library Documentation